

Variáveis e Tipos de Dados Basicos

Objetivo: Declarar variáveis, atribuir valores e verificar seus tipos.

Conceitos: Variáveis, strings, inteiros, floats, booleanos, type().

No terminal Python (>>>)

```
>>> nome = "Alice"
>>> idade = 30
>>> altura = 1.65
>>> eh_estudante = True
```

```
>>> print(nome)
>>> print(idade)
>>> print(altura)
>>> print(eh_estudante)
```

```
>>> type(nome)
>>> type(idade)
>>> type(altura)
>>> type(eh_estudante)
```

=====

O print() mostrará os valores das variáveis.

O type() mostrará as classes dos tipos (ex: <class 'str'>, <class 'int'>).

Operações Aritméticas

Objetivo: Realizar cálculos básicos.

Conceitos: Operadores +, -, *, /, // (divisão inteira), % (módulo), ** (potência).

No terminal Python (>>>)

```
>>> a = 10
```

```
>>> b = 3
```

```
>>> soma = a + b
```

```
>>> print(f"Soma: {soma}") # Usando f-string para formatação
```

```
>>> subtracao = a - b
```

```
>>> print(f"Subtração: {subtracao}")
```

```
>>> multiplicacao = a * b
```

```
>>> print(f"Multiplicação: {multiplicacao}")
```

```
>>> divisao = a / b # Divisão exata (float)
```

```
>>> print(f"Divisão (exata): {divisao}")
```

```
>>> divisao_inteira = a // b
```

```
>>> print(f"Divisão (inteira): {divisao_inteira}")
```

```
>>> resto = a % b
```

```
>>> print(f"Resto da divisão: {resto}")
```

```
>>> potencia = a ** b # 10 elevado a 3
```

```
>>> print(f"Potência (10^3): {potencia}")
```

Manipulação de Strings

Objetivo: Trabalhar com texto.

Conceitos: Concatenação, repetição, indexação, fatiamento (slicing), métodos de string (upper(), lower(), len(), replace(), split()).

No terminal Python (>>>)

```
>>> saudacao = "Olá"
>>> nome_pessoa = "Mundo"

>>> mensagem_completa = saudacao + ", " + nome_pessoa + "!"
>>> print(mensagem_completa)
```

```
>>> repeticao = "Eco! " * 3
>>> print(repeticao)
```

```
>>> palavra = "Python"
>>> print(f"Primeira letra: {palavra[0]}") # Indexação começa em 0
>>> print(f"Última letra: {palavra[-1]}")
>>> print(f"Do segundo ao quarto caractere: {palavra[1:4]}") # Fatiamento [inicio:fim_exclusivo]
```

```
>>> print(f"Em maiúsculas: {palavra.upper()}")
>>> print(f"Em minúsculas: {palavra.lower()}")
>>> print(f"Tamanho da palavra: {len(palavra)}")
```

```
>>> frase = "Eu amo programar em Python."
>>> nova_frase = frase.replace("amo", "adoro")
>>> print(nova_frase)
```

```
>>> palavras_da_frase = frase.split(" ") # Divide a string em uma lista de palavras
>>> print(palavras_da_frase)
```

As strings modificadas ou informações sobre elas.

Listas (Coleções Mutáveis e Ordenadas)

Objetivo: Trabalhar com coleções de itens.

Conceitos: Criar listas, acessar elementos, adicionar (append), remover (pop, remove), tamanho (len), verificar se item existe (in).

No terminal Python (>>>)

```
>>> frutas = ["maçã", "banana", "laranja"]  
>>> print(frutas)
```

```
>>> print(f"Primeira fruta: {frutas[0]}")
```

```
>>> frutas.append("uva") # Adiciona ao final  
>>> print(f"Lista após adicionar uva: {frutas}")
```

```
>>> frutas[1] = "morango" # Modificar um item  
>>> print(f"Lista após modificar banana: {frutas}")
```

```
>>> fruta_removida = frutas.pop(2) # Remove pelo índice e retorna o item  
>>> print(f"Fruta removida (era laranja): {fruta_removida}")  
>>> print(f"Lista após pop: {frutas}")
```

```
>>> frutas.remove("maçã") # Remove pelo valor (primeira ocorrência)  
>>> print(f"Lista após remover maçã: {frutas}")
```

```
>>> print(f"Tem uva na lista? {'uva' in frutas}")  
>>> print(f"Tamanho da lista: {len(frutas)}")
```

Dicionários (Coleções de Pares Chave-Valor)

Objetivo: Trabalhar com dados associativos.

Conceitos: Criar dicionários, acessar valores por chave, adicionar/modificar itens, keys(), values(), items().

```
# No terminal Python (>>>)
```

```
>>> pessoa = {"nome": "Carlos", "idade": 25, "cidade": "São Paulo"}
```

```
>>> print(pessoa)
```

```
>>> print(f"Nome da pessoa: {pessoa['nome']}")
```

```
>>> print(f"Idade: {pessoa.get('idade')}")
```

```
# .get() é mais seguro se a chave não existir
```

```
>>> pessoa["profissao"] = "Engenheiro" # Adicionar novo par
```

```
>>> print(f"Pessoa atualizada: {pessoa}")
```

```
>>> pessoa["idade"] = 26 # Modificar valor existente
```

```
>>> print(f"Pessoa com idade atualizada: {pessoa}")
```

```
>>> print(f"Chaves do dicionário: {list(pessoa.keys())}")
```

```
>>> print(f"Valores do dicionário: {list(pessoa.values())}")
```

```
>>> print("Iterando sobre chave-valor:")
```

```
>>> for chave, valor in pessoa.items():
```

```
...     print(f" {chave}: {valor}")
```

```
... # Pressione Enter novamente para finalizar o bloco for
```

Estruturas de Controle: if-elif-else

Objetivo: Tomar decisões no código.

Conceitos: Condicionais.

```
# No terminal Python (>>>)
```

```
>>> idade_usuario = int(input("Digite sua idade: ")) # input() retorna string, convertemos para int
```

```
>>> if idade_usuario < 18:
```

```
...     print("Você é menor de idade.")
```

```
... elif idade_usuario >= 18 and idade_usuario < 65:
```

```
...     print("Você é adulto.")
```

```
... else:
```

```
...     print("Você é idoso.")
```

```
... # Pressione Enter novamente para finalizar o bloco if
```

Estruturas de Controle: if-elif-else

Objetivo: Tomar decisões no código.

Conceitos: Condicionais.

```
# No terminal Python (>>>)

>>> idade_usuario = int(input("Digite sua idade: ")) # input() retorna string, convertamos para
int

>>> if idade_usuario < 18:
...     print("Você é menor de idade.")
... elif idade_usuario >= 18 and idade_usuario < 65:
...     print("Você é adulto.")
... else:
...     print("Você é idoso.")
... # Pressione Enter novamente para finalizar o bloco if
```

```
# =====
```

Como interagir:

Digite a primeira linha e pressione Enter.

O terminal pedirá: Digite sua idade:. Digite um número e pressione Enter.

Digite as linhas do if, elif e else, prestando atenção na indentação.

O resultado correspondente à idade digitada será impresso.

```
# =====
```

Estruturas de Controle: Loop for

Objetivo: Repetir um bloco de código.

Conceitos: Iterar sobre sequências (listas, strings, range()).

```
# No terminal Python (>>>)

>>> print("Contando de 0 a 4:")

>>> for i in range(5): # range(5) gera números de 0 a 4
...     print(i) # apertar (enter)
...

>>> print("\nFrutas na lista:")

>>> minhas_frutas = ["abacaxi", "manga", "caju"]

>>> for fruta_atual in minhas_frutas:
...     print(fruta_atual)

...

# =====
```


Estruturas de Controle: Loop while

Objetivo: Repetir um bloco de código enquanto uma condição for verdadeira.

Conceitos: Condição de parada, incremento/decremento de contador.

```
# No terminal Python (>>>)
>>> contador = 0
>>> print("Contando com while:")
>>> while contador < 5:
...     print(contador)
...     contador = contador + 1 # ou contador += 1
...
...
```

Definindo e Usando Funções Simples

Objetivo: Criar blocos de código reutilizáveis.

Conceitos: def, parâmetros, return.

```
# No terminal Python (>>>)

>>> def saudar(nome_param):
...     mensagem = f"Olá, {nome_param}! Tudo bem?"
...     return mensagem
...

>>> saudacao_para_ana = saudar("Ana")
>>> print(saudacao_para_ana)

>>> saudacao_para_joao = saudar("João")
>>> print(saudacao_para_joao)

>>> def somar(x, y):
...     return x + y
...

>>> resultado_soma = somar(10, 25)
>>> print(f"Resultado da soma: {resultado_soma}")
```

#-----

Dicas para o Terminal Python:

Setas para Cima/Baixo: Navegam pelo histórico de comandos que você já digitou.

help(funcao_ou_tipo): Para obter ajuda sobre algo (ex: help(str), help(list.append)).

quit() ou exit(): Para sair do interpretador Python.

Indentação: O Python é sensível à indentação (espaços no início da linha). Para blocos como if, for, while, def, as linhas seguintes dentro do bloco devem ser indentadas. O terminal interativo geralmente usa ... para indicar que está esperando a continuação de um bloco.